Self-interpretation in PTS-rk: a rank obstruction and an indexed approach

October 20, 2026

Abstract

We investigate whether the self-interpreter of Matt Brown and Jens Palsberg for System F_ω can be adapted to the original annotated calculus PTS-rk. A direct uniform adaptation is impossible for the fixed-program application interface: application cannot increase the rank of a term's type, whereas closed inhabitants have unbounded type ranks. We prove this obstruction in Rocq. It motivates a family of interpreters indexed externally by rank budgets. Explicit shallow decoders are definable in PTS-rk, and a structural quotation construction works for the independently specified simply typed fragment. Extending it to dependent conversion is an open problem. We separate established theorems, the positive fragment, and the remaining requirements for a full dependent construction.

1 Question and source

The reference is Matt Brown and Jens Palsberg, *Breaking through the Normalization Barrier: A Self-Interpreter for F-omega*, POPL 2016, pp. 5–17, doi:10.1145/2837614.2837623[1]. The supplied PDF has filename `2914770.2837623.pdf`; the theorem labels below refer to its text, not to PDF page indices.

Their target is an **unquoter**: an external quotation operation assigns a typed, beta-normal representation $\ulcorner t \urcorner$ to each term t, and an object-language program recovers t from this representation. The strong version requires reduction to the original term, $\mathsf{unquote}\ulcorner t \urcorner \longrightarrow_\beta^* t$, rather than merely conversion to it. Quotation must preserve syntax injectively, with the paper's precise treatment of equivalent type annotations. Returning the quotation of a normal form is a different operation, called a self-reducer.

Section 3.3, Theorems 3.4–3.5, gives the shallow construction. A neutral variable of type $\forall A : *.A \to A$ blocks each application; substituting the polymorphic identity unblocks it. Sections 4–7 replace this with a deeper representation using typed higher-order abstract syntax, extensional instantiation functions, and interpretations of types. Theorems 6.1–6.3 establish typing, normality, and injectivity up to equivalence of type annotations; Theorem 7.4 establishes strong recovery. Section 10 leaves dependent self-interpretation and self-representation without impredicativity as open directions.

These are not impossibility results from strong normalization alone. The paper specifically explains why a numerical diagonalization argument does not automatically apply to typed quotation. The obstruction below instead uses the exact PTS-rk product rule.

2 The original calculus and the rank invariant

All claims use the existing `PTSrk.Syntax.Term nat n`, including lambda annotations, `Typing.typ`, `Ranks.RankSorts`, and original beta reduction. There is no cumulativity, added

[1] `https://doi.org/10.1145/2837614.2837623`

primitive, eta conversion, or replacement typing judgment. Its sort and product rules are

$$\Gamma \vdash \mathcal{U}_i : \mathcal{U}_0, \qquad \frac{\Gamma \vdash A : \mathcal{U}_a \quad \Gamma, x : A \vdash B : \mathcal{U}_b}{\Gamma \vdash \Pi x : A.B : \mathcal{U}_{\max(a+1,b)}}.$$

The **type rank** of a typed term t is r when $\Gamma \vdash t : T$ and $\Gamma \vdash T : \mathcal{U}_r$. This is the rank of its type, not its largest sort literal or a bound on all redexes inside t. In well-formed contexts it exists, is unique, and is invariant under beta conversion of typed terms. These facts are already proved in `NormalizationComplexity/RedexRanks.v`.

Lemma 2.1 (Application rank)**.** *If Γ is well formed and $f\ u$ is typed, then $\mathrm{rk}_\Gamma(f\ u) \le \mathrm{rk}_\Gamma(f)$.*

Proof. Invert the typing of the application to obtain $f : \Pi x : A.B$ and $u : A$. If $A : \mathcal{U}_a$ and $B : \mathcal{U}_b$ under the binder, the type of f has rank $\max(a+1, b)$. Substitution preserves $B : \mathcal{U}_b$, so $B[u/x]$ has rank b. Uniqueness of type rank removes any dependence on the chosen derivation or intervening conversion rules. □

Consequently, for any well-typed finite application spine,

$$\mathrm{rk}(E\, a_1 \cdots a_m) \le \mathrm{rk}(E).$$

Arguments may be types, representations, or other terms. Their number and their construction may depend arbitrarily on the represented program. This bound already holds without a computable quotation operation.

Theorem 2.2 (No universal fixed-head evaluator)**.** *There is no closed typed PTS-rk term E such that every closed typed term t is beta-convertible to a well-typed finite application spine headed by E.*

Proof. Define closed types and inhabitants by

$$A_0 = \mathcal{U}_0, \quad A_{r+1} = \Pi x : A_r.\mathcal{U}_0, \qquad v_0 = \mathcal{U}_0, \quad v_{r+1} = \lambda x : A_r.\mathcal{U}_0.$$

The original rules derive $A_r : \mathcal{U}_r$ and $v_r : A_r$ for every r. Suppose E has type rank R. An application spine of E has rank at most R, so invariance under conversion prevents it from being convertible to v_{R+1}. This contradicts universality. □

The Rocq declaration is `RankObstruction.no_universal_fixed_head_evaluator`. Its specification allows arbitrary externally chosen arguments and arbitrary intermediate result types. It assumes neither an encoding nor normality nor injectivity, making the exclusion stronger than failure of one particular encoding.

This excludes a direct port of the actual fixed unquote programs in Sections 3.3 and 7, including implicit type applications. A fixed wrapper chosen once is another fixed E and is excluded as well. Section 3.2 also permits implicit type *abstractions*. If interpreted as arbitrarily many term-dependent outer wrappers, these lie outside the application-spine theorem: external abstraction can raise rank. We do not claim to have excluded every liberal interpretation of that allowance, externally generated interpreter families, or a weaker observational semantics.

3 What the sort cycle does and does not provide

Every $\mathcal{U}_r$ has type $\mathcal{U}_0$, but this does not place every type in $\mathcal{U}_0$. For example $A \to A$ has rank $r+1$ when $A : \mathcal{U}_r$. Thus there is no single sort playing the role of the arrow-closed kind $*$ of System F_ω. Replacing every occurrence of $*$ by $\mathcal{U}_0$ in the paper's code does not produce a typing derivation.

There is, however, an identity at every external rank:

$$\mathsf{I}_r = \Pi A : \mathcal{U}_r.\, A \to A, \qquad \mathsf{id}_r = \lambda A : \mathcal{U}_r.\lambda x : A.x, \qquad \mathsf{I}_r : \mathcal{U}_{r+1}, \quad \mathsf{id}_r : \mathsf{I}_r.$$

The numerical index r is part of generated syntax. Rocq quantification over $r : \mathbb{N}$ produces a family of original PTS terms; it is not an internal PTS quantifier over ranks. In particular an object numeral cannot simply be substituted for the sort label in $\mathcal{U}_r$.

For a blocker rank r and an output-type rank s, define

$$\begin{aligned} \mathsf{Code}_r(A) &= \mathsf{I}_r \to A, \\ \mathsf{D}_{r,s} &= \lambda A : \mathcal{U}_s.\lambda q : \mathsf{Code}_r(A).\, q\, \mathsf{id}_r, \\ T_{r,s} &= \Pi A : \mathcal{U}_s.\, \mathsf{Code}_r(A) \to A. \end{aligned}$$

The exact original judgments, proved in `RankedDecoder.v`, are

$$\begin{aligned} A : \mathcal{U}_s \implies \mathsf{Code}_r(A) &: \mathcal{U}_{\max(r+2,s)}, \\ \mathsf{D}_{r,s} : T_{r,s}, \qquad T_{r,s} &: \mathcal{U}_{\max(r+3,s+1)}. \end{aligned}$$

For any typed code $q : \mathsf{Code}_r(A)$, $\mathsf{D}_{r,s}\, A\, q \longrightarrow_\beta^* q\, \mathsf{id}_r$. The raw packet $p_r(t) = \lambda i : \mathsf{I}_r.t$ (with the necessary de Bruijn shift) satisfies $\mathsf{D}_{r,s}\, A\, p_r(t) \longrightarrow_\beta^* t$ in three beta steps. This is a typed retraction for every original typed term, including dependent terms. It is **not a quotation theorem**: $p_r(t)$ retains every source redex. The declaration `typed_ranked_retraction` deliberately makes only the retraction claim.

The theorem `decoder_type_not_in_result_universe` formalizes a failure of self-specialization. The decoder cannot be instantiated at its own type in its output universe: $T_{r,s}$ has rank at least $s+1$, hence cannot also inhabit $\mathcal{U}_s$. Increasing an external index is therefore a real change of program, not the paper's self-application of one polymorphic interpreter.

4 A structural quotation route for the simply typed fragment

The existing independent source `ChurchSTLC` has a distinguished ground type variable $o : \mathcal{U}_0$, simple types σ, and annotated lambda terms. It has its own typing rules. Its embedding uses

$$|o| = o, \qquad |\sigma \to \tau| = |\sigma| \to |\tau|, \qquad \mathrm{ord}(\sigma \to \tau) = \max(\mathrm{ord}(\sigma) + 1, \mathrm{ord}(\tau)).$$

All type codes are beta-normal, and $|\sigma|$ inhabits $\mathcal{U}_{\mathrm{ord}(\sigma)}$ in the target context. No source rule depends on conversion of terms inside types.

Fix a budget d bounding the order of the **function type of every application** in a source typing derivation. It is not enough to bound the final result type. For an example entirely

within STLC, put $\sigma_0 = o \to o$, $w_0 = \lambda z : o.z$, $\sigma_{N+1} = \sigma_N \to \sigma_0$, and $w_{N+1} = \lambda x : \sigma_N.w_0$. Then $(\lambda x : \sigma_N.w_0)\, w_N$ has the fixed result type σ_0, while its function and argument orders grow without bound. The independent relation `bounded_type` adds only this numerical bound to the original STLC rules; `bounded_type_source` forgets it, and `every_source_has_budget` proves that every source derivation has some finite budget. No quotation or recovery property defines the source domain.

Introduce variables $b_0 : \mathsf{I}_0, \ldots, b_d : \mathsf{I}_d$. The prequotation is the following derivation-directed structural translation:

$$\begin{aligned} q(x) &= x, \\ q(\lambda x : |\sigma|.t) &= \lambda x : |\sigma|.q(t), \\ q(f\, u) &= b_r\, |\sigma \to \tau|\, q(f)\, q(u), \qquad r = \mathrm{ord}(\sigma \to \tau), \end{aligned}$$

where the last case uses the source judgment $f : \sigma \to \tau$. The formal version supplies all scope changes and binder shifts explicitly. No application in $q(t)$ has a lambda at its head: each translated application is headed by a neutral blocker. The remaining annotations are normal simple-type codes. Thus the quotation is normal for the **full original PTS reduction**, including annotation reduction.

Typing follows from $b_r\, |\sigma \to \tau| : (|\sigma| \to |\tau|) \to (|\sigma| \to |\tau|)$ and the translated subderivations. Substituting id_r for each blocker gives strong recovery to the original annotated source term by induction. Syntactic injectivity is proved at each fixed budget with a structural eraser for the administrative blocker pattern. Recovery up to beta conversion alone would not establish injectivity, since distinct source terms may be beta-convertible.

The Rocq module `RankedQuotation` implements these claims for arbitrary bounded derivations and explicitly typed blocker contexts. Its existence theorem is relational, in Rocq `Prop`; it is not an extracted executable quoter. The structural clauses nevertheless give the intended external algorithm. The theorem `finite_budget_quotation` supplies the actual finite blocker context, and `finite_package_injective` proves injectivity of the resulting packages. `every_source_has_normal_representation` covers every independent STLC derivation at some budget.

The finite-telescope interface abstracts those blockers in descending order:

$$\begin{aligned} \mathsf{Code}_{\le d}(A) &= \Pi b_d : \mathsf{I}_d. \cdots \Pi b_0 : \mathsf{I}_0.A, \\ \ulcorner t \urcorner_d &= \lambda b_d : \mathsf{I}_d. \cdots \lambda b_0 : \mathsf{I}_0.q(t), \\ \mathsf{D}^{\mathrm{tel}}_{d,s} &= \lambda A : \mathcal{U}_s.\lambda q : \mathsf{Code}_{\le d}(A).q\, \mathsf{id}_d \cdots \mathsf{id}_0. \end{aligned}$$

For $A : \mathcal{U}_s$, direct rank calculation gives $\mathsf{Code}_{\le d}(A) : \mathcal{U}_{\max(d+2,s)}$ and a decoder type of rank $\max(d+3, s+1)$. `TelescopeDecoder.family_decoder_typing` checks the actual decoder, with its exact rank expressed by the recursive function `package_rank`. The theorem `fixed_decoder_bounded_quotation` uses the same decoder for every source with budget d and result order s.

Closing over the distinguished ground type also has an internal solution. Let

$$\begin{aligned} K_s &= \Pi o : \mathcal{U}_0.\mathcal{U}_s, \\ C_d(P) &= \Pi o : \mathcal{U}_0.\mathsf{Code}_{\le d}(P\, o), \\ \mathsf{D}^{\mathrm{cl}}_{d,s} &= \lambda P : K_s.\lambda q : C_d(P).\lambda o : \mathcal{U}_0.(q\, o)\, \mathsf{id}_d \cdots \mathsf{id}_0. \end{aligned}$$

This is one closed PTS term for each fixed pair (d, s), with type $\Pi P : K_s.C_d(P) \to (\Pi o : \mathcal{U}_0.P\,o)$. For a term-closed source $t : \sigma$ with its single free type variable o, put $P_\sigma = \lambda o : \mathcal{U}_0.|\sigma|$ and $Q = \lambda o : \mathcal{U}_0.\ulcorner t \urcorner_d$. The closed representation Q is normal, and the actual decoder application satisfies

$$\mathsf{D}^{\mathrm{cl}}_{d,s}\,P_\sigma\,Q \longrightarrow^*_\beta \lambda o : \mathcal{U}_0.t, \qquad s = \mathrm{ord}(\sigma).$$

`ClosedDecoder.closed_fixed_decoder_bounded_quotation` proves original typing and strong recovery for this closed interface. The ground-type abstraction is inside the decoder; it is not an external evaluation step. Every independent STLC source derivation has a finite budget, so this is an infinite fragment result. The fragment does not include all PTS-rk syntax or the decoder's own polymorphic code, and no self-applicability claim follows.

5 Why dependent quotation is still open

The rank-indexed identities solve a rank-selection problem. They do not by themselves solve dependent conversion. Consider the well-formed context

$$A : \mathcal{U}_0, \quad B : \Pi x : A.\mathcal{U}_0, \quad a : A, \quad f : \Pi x : A.B\,x.$$

The source term $f\,((\lambda x : A.x)\,a)$ can be given type $B\,a$. If its inner redex is replaced by $b_1\,(A \to A)\,(\lambda x : A.x)\,a$, that argument is neutral, so its occurrence in the result type is no longer convertible to a. Blocking other applications as well does not restore the lost conversion. This example diagnoses failure of the naive, type-preserving shallow translation; it is not a theorem that every dependent encoding is impossible.

The paper's Figure 2 term-reduction rules do not reduce inside type annotations; PTS-rk's original reduction does. Its normality theorem therefore cannot be imported as the stronger annotation-normality result required here. One cannot avoid the dependent conversion issue by leaving all type expressions untouched: original PTS-rk reduction traverses lambda annotations and products, so a reducible annotation would prevent full normality of the quote. Nor may the typing theorem be replaced by a native Rocq checker judgment, or supplied as a premise of the desired quotation theorem.

A deeper indexed construction would need jointly compatible translations of contexts, types, terms, substitution, and conversion. An explicit interpretation relation or transported representation of equality might help, but the required witnesses must be built inside the original PTS. The extensional instantiation functions of Brown–Palsberg suggest a direction; their correctness for F_ω does not supply this dependent coherence theorem.

6 Formalization status and next obligations

The separate library is `PTSrk.SelfInterpreter`, physically in `theories/self_interpreter`. It imports the original kernel and existing normalization infrastructure; neither arithmetic interpretation nor either set-model library is a dependency. The reference ZFC development is unchanged.

The formalized obstruction is `RankObstruction.no_universal_fixed_head_evaluator`. Explicit inhabitants at every type rank are `rank_type_sort` and `rank_value_typing`. The decoder family and its exact rank are `RankedDecoder.decoder_typing` and `decoder_type_sort`;

`typed_ranked_retraction` states its restricted positive contract. `RankedQuotation` supplies the separate STLC quotation development, and `TelescopeDecoder` and `ClosedDecoder` make its interpreters actual object-language programs. The development README records the exact public endpoints and proof boundary.

The development uses the pinned Rocq 9.2.0, Stdlib 9.1.0, Equations 1.3.2+9.2, OCaml 4.14.2, and Dune 3.23.1 environment. Fresh assumption reports and recursive kernel checks are compared with a pristine upstream build. No new axioms, admitted proofs, unsafe casts, modified PTS rules, or disabled checks are part of this development. Verification commands and failures are recorded separately from theorem claims.

The remaining research programme is:

1. Make derivation-directed quotation executable, prove its relationship to the relational construction, and retain syntax-level injectivity.
2. Define an independently specified bounded dependent source fragment and solve its substitution and conversion coherence obligations.
3. Prove all five required contracts for that dependent fragment: original typing, full normality, injectivity under a stated annotation equality, strong recovery, and coverage of every intended source derivation at some finite external budget.
4. Investigate representations supporting additional operations such as syntax inspection. A shallow recovery theorem alone is not a deep encoding.

No full dependent self-interpreter or all-rank single program is claimed. The positive result is an indexed fragment construction, alongside a precise obstruction explaining why external rank indices are necessary for the fixed-head interface.

To rebuild this note, run `bash tools/build-self-interpreter-note.sh`. To compile the development in the pinned environment, run `dune build @theories/self_interpreter/audit`; use a fresh absolute build directory for fresh assumption output.